car-diagnostic-analyser

Browser-based, read-only car diagnostics over a USB OBD-II adapter — with a live view of every check as it runs.

Design document · v0.1 Author: Serkan Yucel Status: Draft 08 November 2026

1. Goals and scope
2. How it connects
3. What can be checked
4. Safety: read-only by construction
5. Architecture
6. Scan engine and progress
7. Screens
8. Data model and storage
9. Tech stack and repo layout
10. Testing and the demo car
11. Roadmap
12. Risks and open questions

1 Goals and scope

Plug a USB OBD-II adapter into the car and a laptop, open the app in Chrome, press **Run scan**, and watch a top-down car diagram light up system by system while a counter shows exactly how many checks have run and what each one returned. Finish with a report that separates what the car *told us* from what *needs a human to look at*.

It's a personal project that starts with the generic OBD-II basics and grows one vehicle system at a time.

Goals

- One-click full scan of everything the connected car exposes
- Honest progress: "23 of 41 checks · ABS timed out", never a fake health %
- Fault codes with plain-English meaning and freeze-frame context
- Live readings (RPM, coolant, battery voltage, fuel trims …)
- Maintenance and inspection log for things no ECU reports (pads, oil, filters, tyre gauge readings)
- Scan history, compare two scans, export a PDF report
- A demo car so anyone can try it without hardware

Non-goals (for now)

- Clearing fault codes, actuator tests, coding, adaptations, key programming
- Mobile browsers and Bluetooth adapters (Web Serial is desktop Chrome/Edge)
- Coverage of every make — profiles are added one car at a time
- Cloud accounts or sync; all data stays in the browser
- Diagnosing *why* beyond well-known common causes

2 How it connects

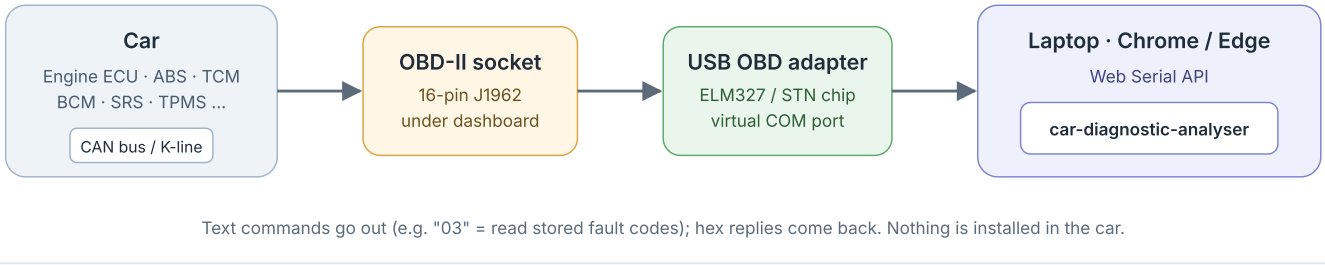


Figure 1 — Car diagnostic socket → USB adapter → laptop → browser app.

Every car sold in the EU since 2001 (petrol) / 2004 (diesel) has a standard 16-pin diagnostic socket (EOBD, the European equivalent of OBD-II), usually under the dashboard on the driver's side. A USB adapter built on an ELM327-compatible chip (genuine ELM327, or the faster STN chips in OBDLink devices) translates simple text commands into the car's bus protocol and shows up on the laptop as a virtual COM port. The browser talks to that port through the **Web Serial API**.

LAYER	CHOICE FOR V1	WHY
Adapter	OBDLink EX (USB, STN chip, FTDI serial) — any ELM327-compatible USB adapter should work	Reliable, fast, well documented; cheap ELM327 clones often have buggy firmware
Browser	Chrome or Edge, desktop (Windows first)	Only Chromium desktop ships Web Serial; requires HTTPS or `localhost`
Car protocol	Auto-detect (`ATSP0`); expect ISO 15765-4 CAN on most post-2008 cars	The adapter handles CAN, KWP2000 and ISO 9141 for us
Serial settings	Try 115200 then 38400 baud, 8N1	STN adapters default high; many ELM327 clones use 38400

3 What can be checked

Generic OBD-II only guarantees emissions-related data from the engine computer. Everything else (ABS, airbags, gearbox, body, tyre pressures, oil life) lives in other modules that need manufacturer-specific requests. The app treats each check's **source** as a first-class field so the report never implies more than it knows.

SYSTEM	CHECKS	SOURCE	PHASE
Engine	Stored / pending / permanent fault codes, check-engine light, freeze frame, distance with MIL on, distance since codes cleared	Generic services 01, 02, 03, 07, 0A	1–6
Emissions	Readiness monitors (catalyst, O_2 sensors, EVAP, EGR, misfire, fuel system …) — useful before an MOT	Generic 01 01, 01 41	5

Live engine data	RPM, speed, coolant, intake air, load, throttle, MAF/MAP, fuel trims, O_2 sensors, timing, run time	Generic service 01 (where supported)	7
Battery / charging	Module voltage (01 42) or adapter pin voltage (`ATRV`); cranking dip, charging voltage at idle	Generic / adapter	3
Oil	Oil temperature (01 5C, rarely supported); oil-life % and service-due counters	Generic, rare / Profile	7 / 16
Vehicle identity	VIN, calibration IDs, ECU name	Generic service 09	3
ABS / brakes	ABS module fault codes, wheel-speed sensors, pad-wear warning (if the car has wear sensors)	Profile UDS 19 / 22	14
Tyres	Per-wheel pressure & temperature on cars with *direct* TPMS; warning-only on *indirect* TPMS (uses ABS wheel speeds, no pressures exist)	Profile	15
Gearbox, airbags, body	Module fault codes, gearbox fluid temperature	Profile	later
Things no ECU knows	Brake pad / disc thickness, brake fluid age, oil level & condition, air / cabin / fuel filters, tyre tread depth, wipers, lights	Manual inspection log + service intervals	12

4 Safety: read-only by construction

A diagnostic adapter can send commands that clear codes, run actuator tests, reset modules or rewrite settings. This app is a reader, and that is enforced in code, not by convention.

> **Rule:** every byte sequence sent to the car passes through a single `ReadOnlyGuard.check(request)` . Anything not on the allowlist throws `ForbiddenRequestError` before it reaches the serial port. There is no “advanced mode” switch.

ALLOWED	BLOCKED (ALWAYS)
OBD-II 01 current data · 02 freeze frame · 03 stored codes · 06 test results · 07 pending codes · 09 vehicle info · 0A permanent codes UDS 19 read DTC info · 22 read data by ID · 3E tester present · 10 01 default session only Adapter config: `ATZ ATE0 ATL0 ATS0 ATH1 ATSP0 ATDPN ATRV ATI ATSH ATCRA ATST ATAT`	OBD-II 04 clear codes · 08 on-board control UDS 11 ECU reset · 14 clear DTCs · 27 security access · 28 comms control · 2E write data · 2F I/O control · 31 routines · 34–37 up/download · 3D write memory · 85 DTC setting · 10 02/03 non-default sessions Any unknown AT/ST command, raw `ATMA` bus monitor

Operational rules shown on the connect screen: car parked, handbrake on, ignition on; never scan while driving; unplug the adapter afterwards (some drain the battery). If the battery is below 11.8 V the app warns before starting. Full list in `design/safety-allowlist.md`.

5 Architecture

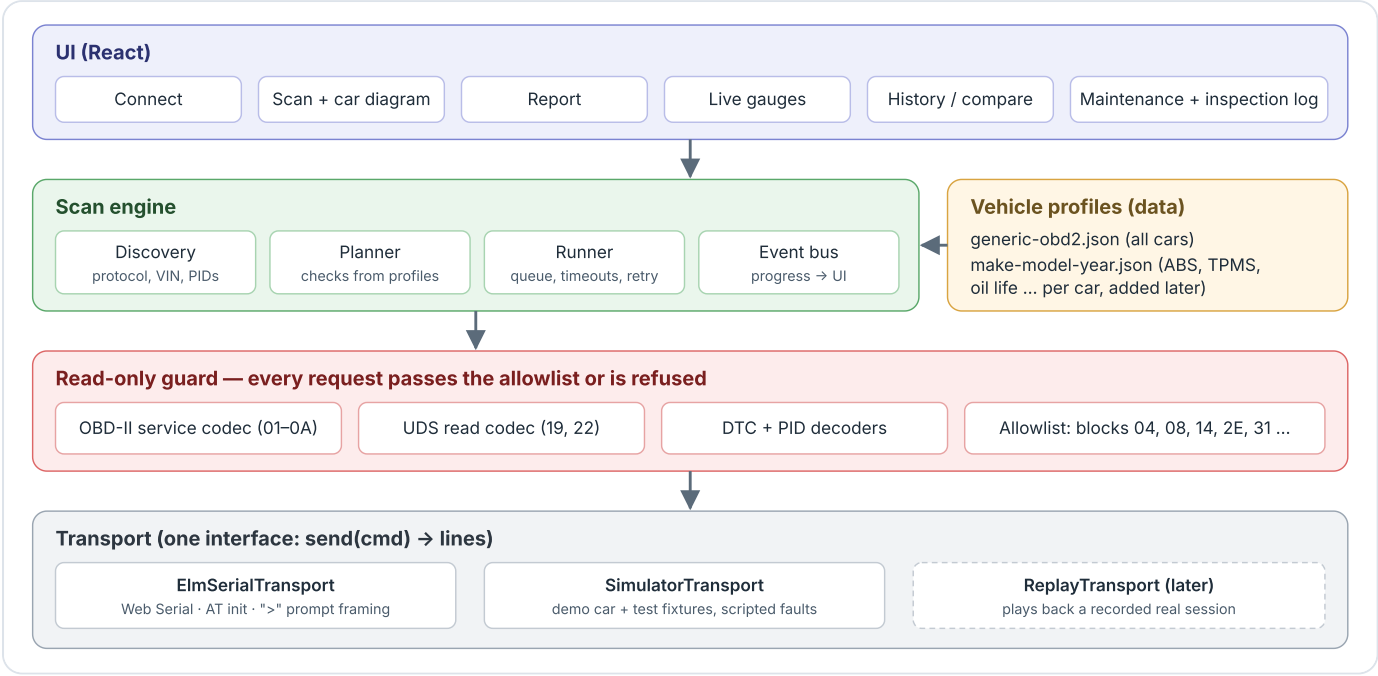


Figure 2 — Layers. The UI never talks to the transport directly; everything goes through the scan engine and the read-only guard.

Transport

- One interface: `open()`, `send(cmd, timeoutMs) → string[]`, `close()`
- **ElmSerialTransport**: Web Serial, line framing on the `>` prompt, one command in flight at a time
- **SimulatorTransport**: answers like an ELM327 from a scenario file, with configurable latency and failures

Protocol

- Builds requests (`0105`, `1902FF`) and parses replies, including multi-frame ISO-TP responses and multiple ECUs answering
- Maps adapter errors (`NO DATA`, `UNABLE TO CONNECT`, `CAN ERROR`, `?`) to typed results
- Decoders are pure functions from bytes to values — easy to unit test

Scan engine

- **Discovery**: protocol, VIN, which PIDs the car supports (bitmaps 0100/0120/0140...), which modules answer
- **Planner**: turns discovery + profiles into an ordered list of checks, so the total is known before running

Profiles (data, not code)

- `generic-obd2.json` applies to every car
- Per-vehicle JSON adds modules (CAN header, e.g. ABS `7B0/7B8`), DIDs and decoders by name
- Selected by VIN prefix (WMI + model year) or picked manually

- **Runner**: executes sequentially, emits events, handles timeout/retry/cancel

6 Scan engine and progress

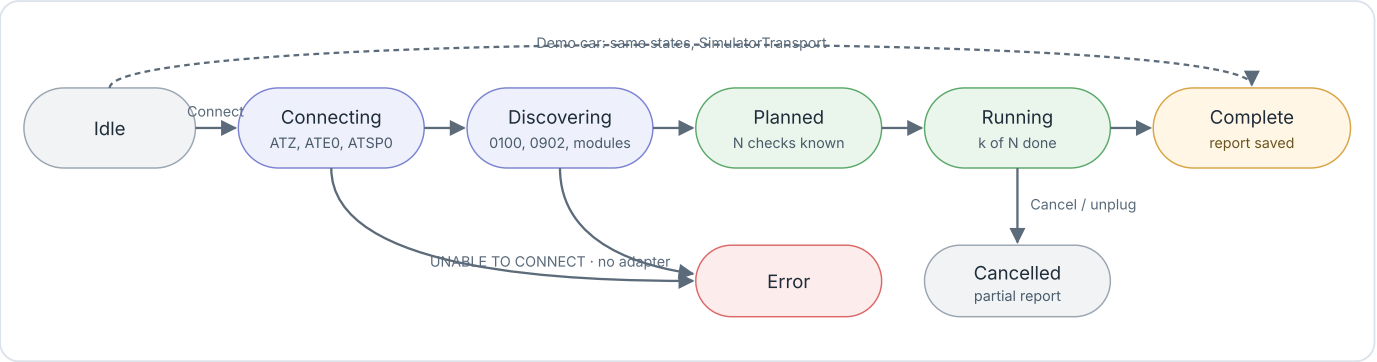


Figure 3 — Scan lifecycle. Cancel or unplug at any time yields a partial report, never a lost one.

What the progress bar counts

The total **N** is fixed once planning finishes, so the counter only ever goes up: “k of N checks”. A check is one question to one module (e.g. “Engine ECU · stored fault codes”). The bar is segmented by result so the colour mix itself tells the story.

RESULT STATUS	MEANING	SHOWN AS
Running	Request sent, waiting for reply	Pulsing zone on the car diagram
Reading received	Valid answer, nothing abnormal	Green
Fault reported	Codes present, MIL on, or value outside its normal range	Amber
Unsupported	Car says it doesn't support this (bitmap or `NO DATA` / negative response 0x11/0x12/0x31)	Grey, excluded from “coverage”
No response	Timed out after retries, or bus error — *we don't know*	Red outline; never treated as “OK”
Manual inspection	Can't be read electronically; pulled from the inspection log or flagged as to-do	Purple

> “ABS · no response” and “ABS · checked, 0 faults” are different outcomes and are always worded differently in the UI and the report.

Events emitted to the UI

```
scan:planned   { total, checks: CheckPlan[] }
check:start    { checkId, system, label, request, ecu }
check:result   { checkId, status, value?, dtcs?, raw, durationMs }
live:sample    { pid, value, unit, t }
scan:done      { scanId, summary }        // also on cancel → partial: true
scan:error     { stage, message, recoverable }
```

Timing

Requests are sequential (the adapter handles one at a time). Default timeout 1 s for engine requests, 2 s for other modules, one retry, adaptive timing (`ATAT1`). A generic scan of ~40 checks should finish in under a minute on CAN; older K-line cars are slower and the ETA adapts from measured durations.

7 Screens

7.1 Connect

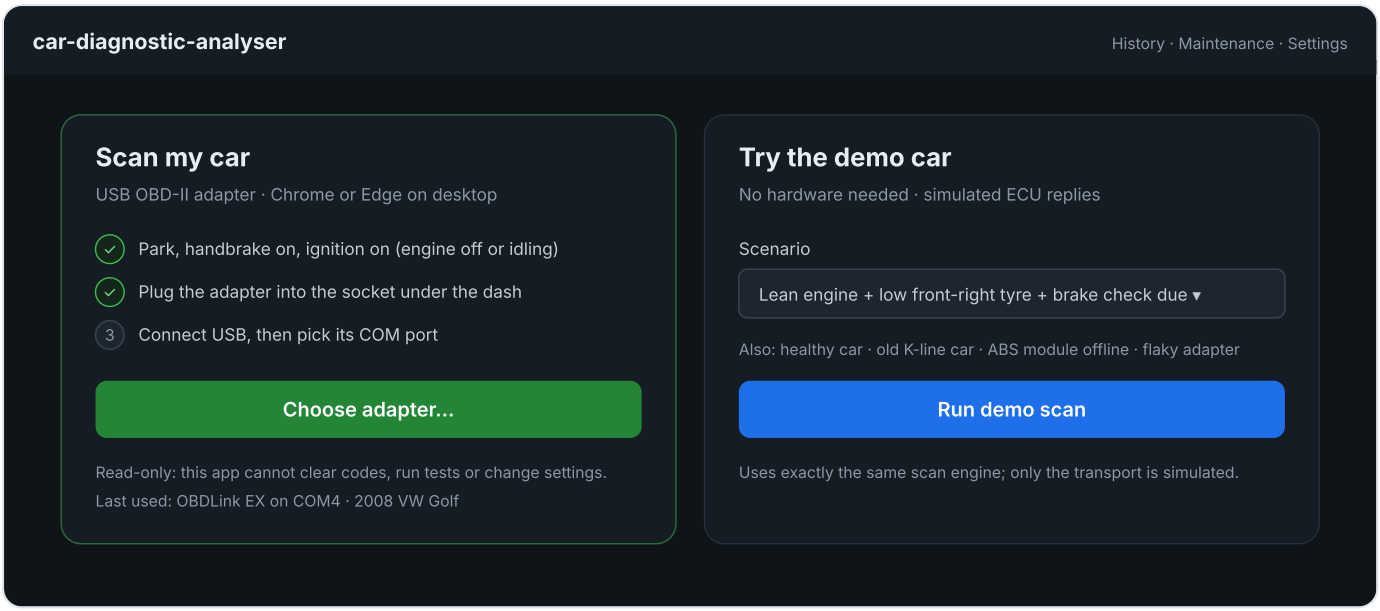


Figure 4 — Two equal entry points: the real car via USB, or the demo car with selectable scenarios.

7.2 Scan in progress

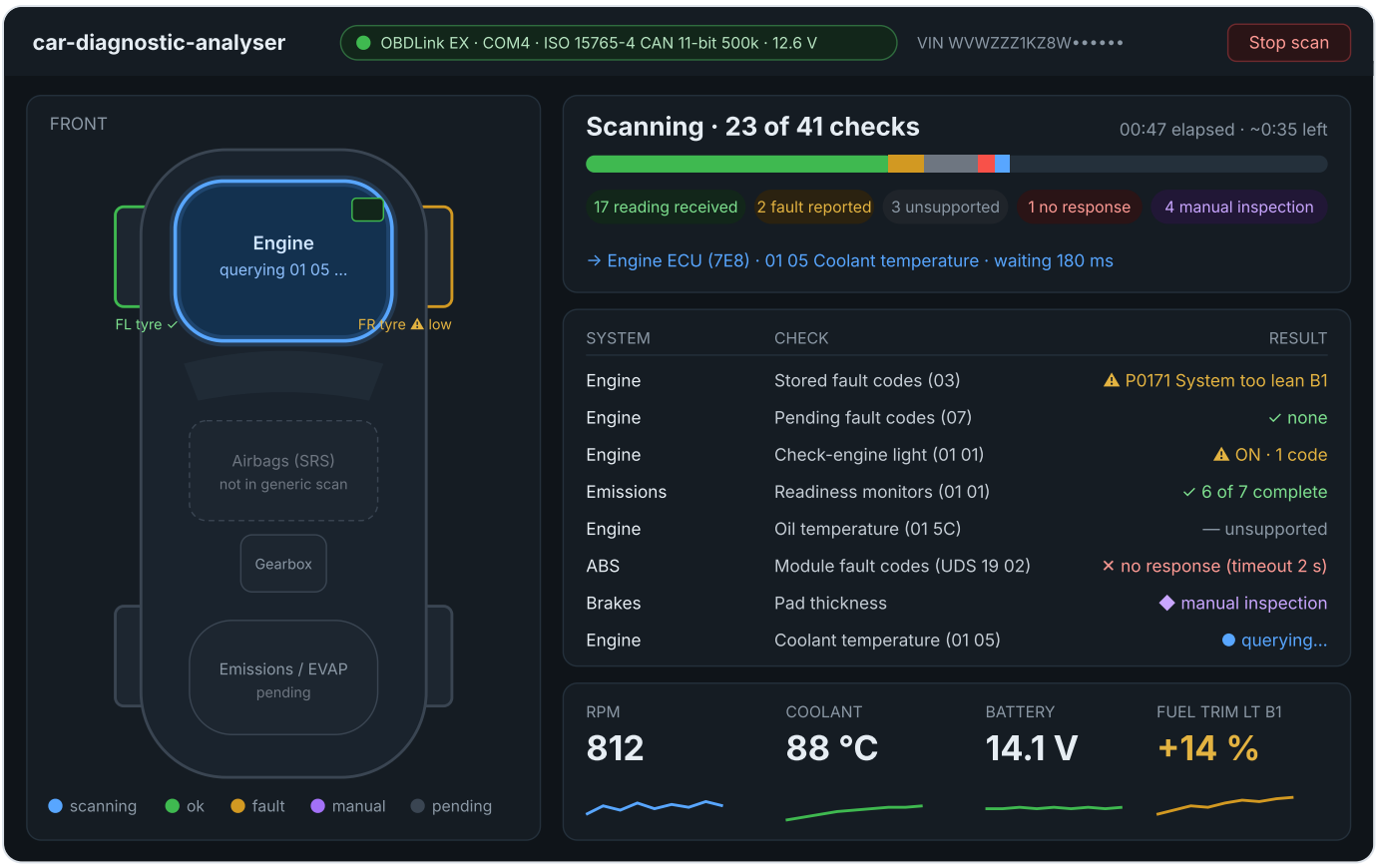


Figure 5 — The car diagram highlights the zone being queried; the counter, segmented bar, status chips and check list update per event; a live strip shows streaming readings.

Car diagram zones: engine bay, battery, four wheels (tyre + brake corner each), cabin (airbags, body), gearbox, rear (emissions/EVAP). Each zone takes the “worst” status of its checks. Zones with no checks for this car are drawn dashed with a short reason (“not in generic scan”). Clicking a zone filters the check list.

7.3 Report

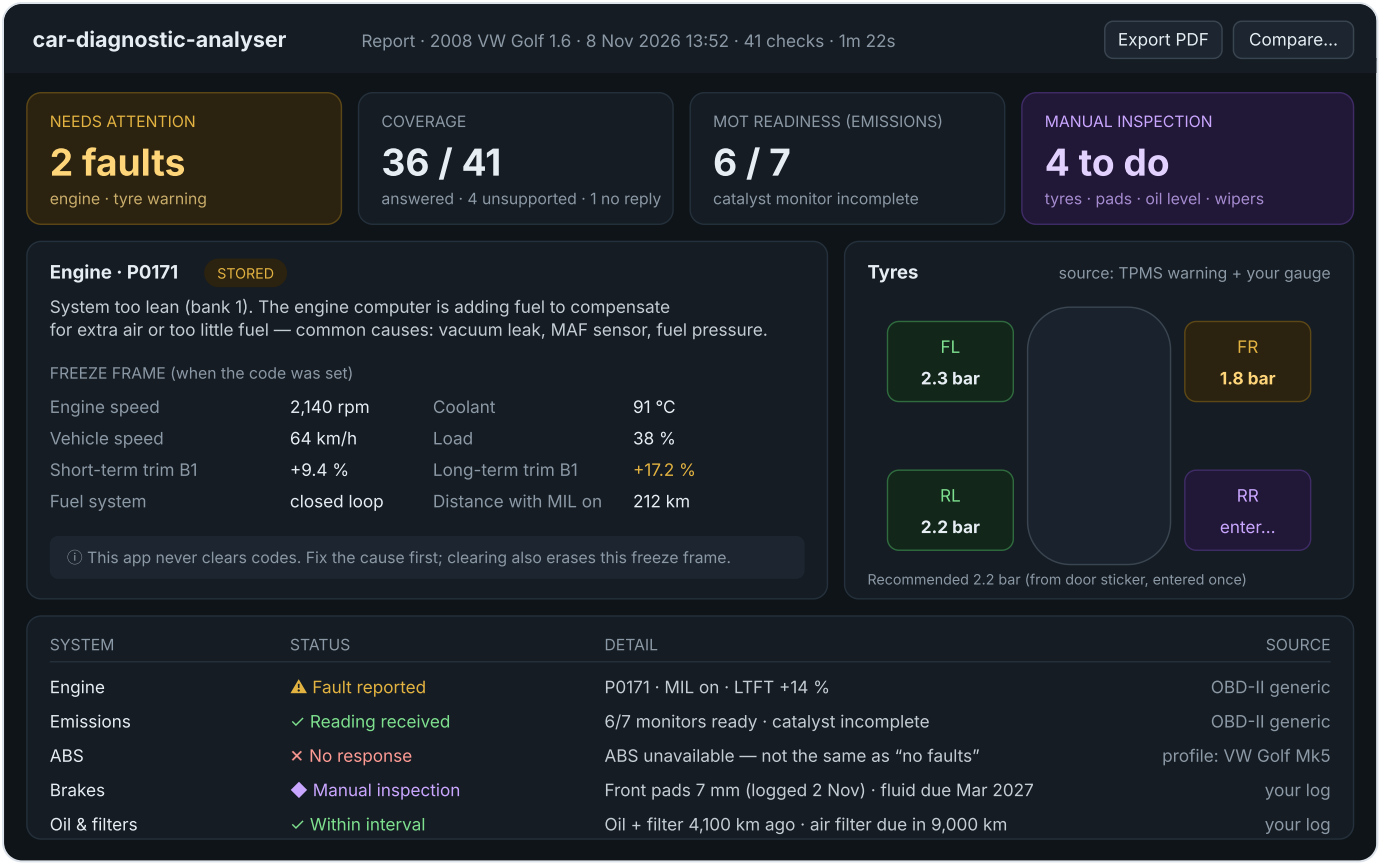


Figure 6 — Summary tiles, fault detail with freeze frame and plain-English explanation, tyre panel mixing car data and gauge entries, and a per-system table that always names its source.

7.4 Live data

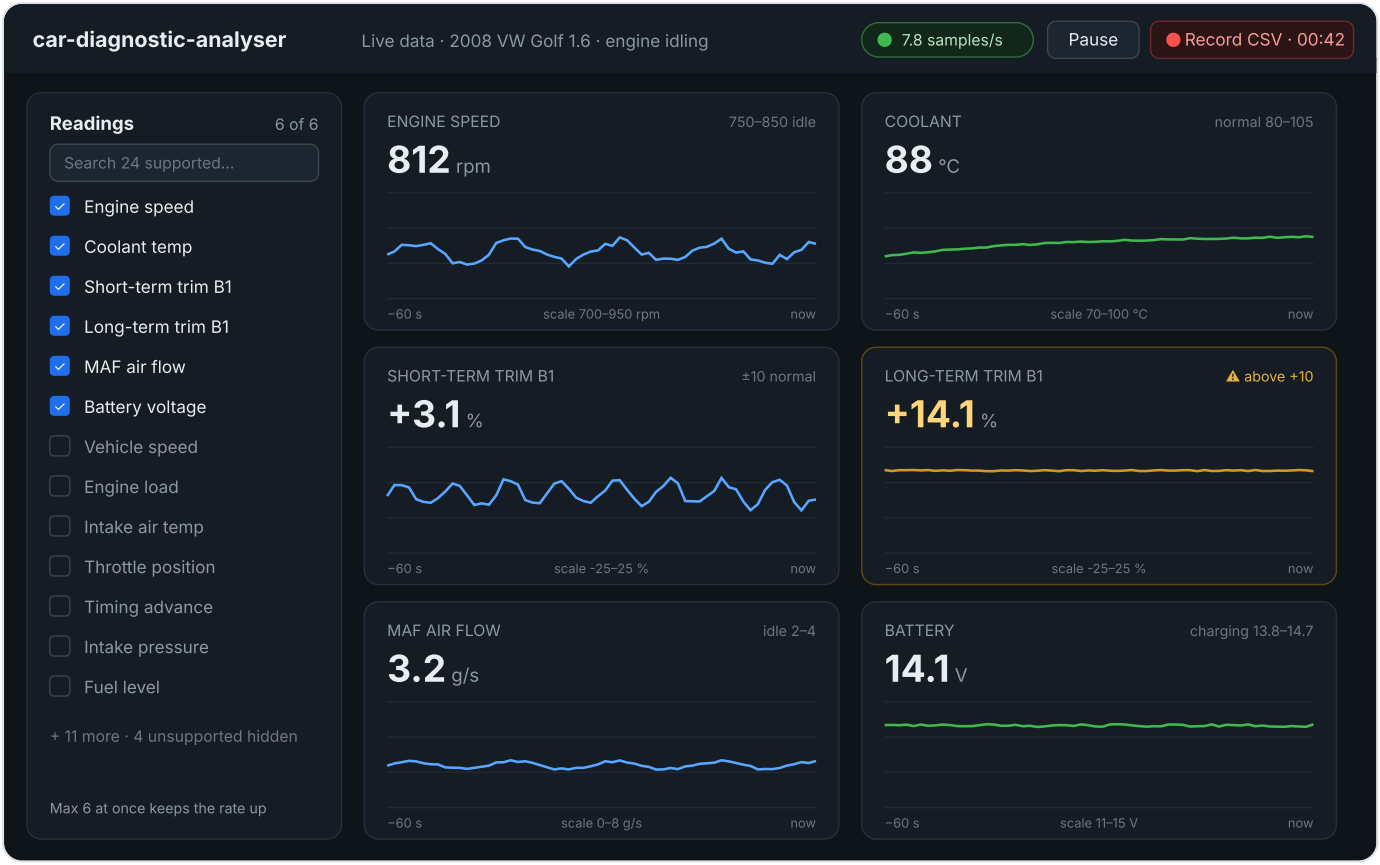


Figure 7 — Pick up to six supported readings; each gets a 60-second chart with its current value and normal range. Out-of-range readings (here long-term fuel trim) are highlighted. Record writes the samples to CSV.

Only PIDs the car reported as supported are offered. On CAN cars several PIDs are requested in one message to keep the sample rate up; the rate is shown in the top bar so a slow adapter is obvious.

7.5 History and compare

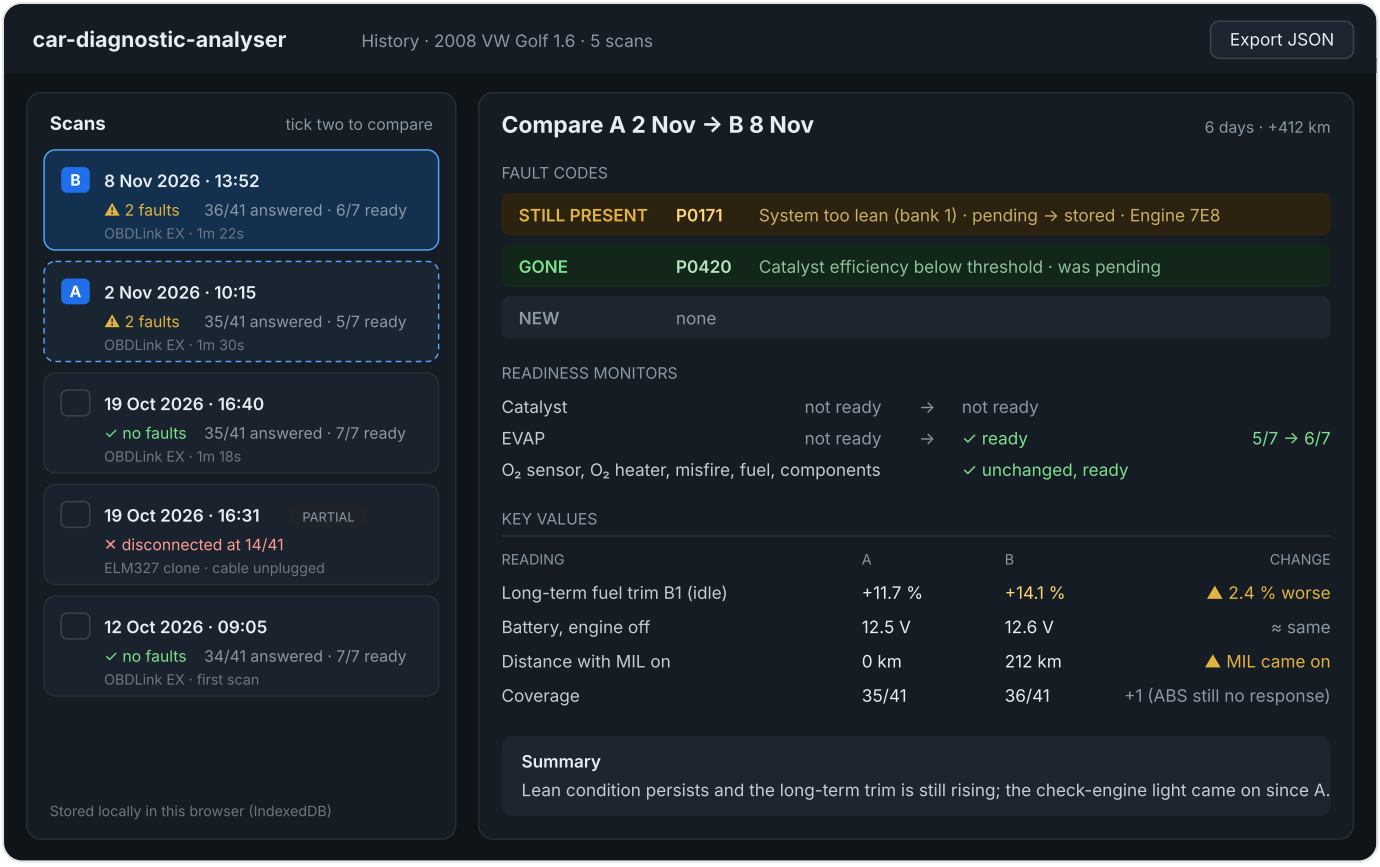


Figure 8 — Saved scans per vehicle (partial scans are marked). Tick two to compare: fault codes new / still present / gone, readiness changes and key value deltas, with a one-line summary.

7.6 Maintenance and inspection

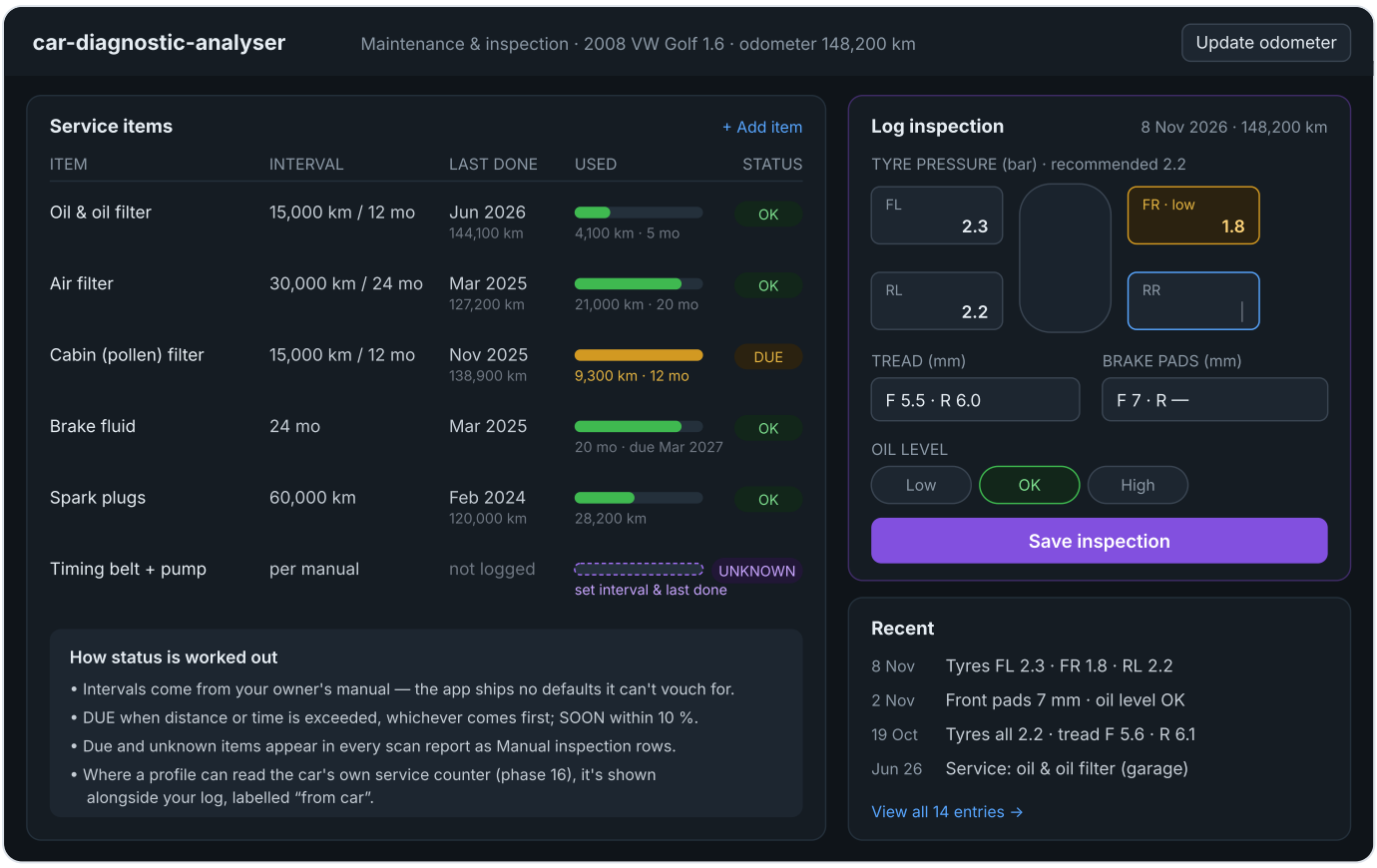


Figure 9 — Service items tracked by distance and/or time from the owner's manual, and a quick inspection form for tyre pressures, tread, pads and oil level. Due, unknown and recent items feed every scan report as Manual inspection rows.

7.7 Vehicles and settings

- **Vehicles**: VIN, nickname, profile, recommended tyre pressures (from the door-pillar sticker), odometer.
- **Settings**: units (°C / bar / km by default, psi and miles optional), preferred adapter baud rate, VIN masking in exports, JSON backup / restore.

---

8 Data model and storage

All data is stored locally in IndexedDB. Each scan stores the raw request/response log alongside decoded results, so decoders can be improved and old scans re-decoded later.

```
Vehicle      { id, vin?, nickname, make?, model?, year?, profileId, tyreSpec?,
odometerKm? }
Scan         { id, vehicleId, startedAt, finishedAt, adapter, protocol, partial,
summary }
CheckResult  { scanId, checkId, system, label, source, status, value?, unit?,
dtcs?, raw[], durationMs }
Dtc          { code: "P0171", kind: stored|pending|permanent, ecu, description,
freezeFrame? }
```

```
LiveSession  { id, vehicleId, pids[], samples: [t, pid, value][] }
ServiceItem  { id, vehicleId, kind, intervalKm?, intervalMonths?, lastDoneAt?,
lastDoneKm? }
Inspection   { id, vehicleId, at, kind, wheel?, valueMm?|valueBar?, note? }
```

Full field definitions and IndexedDB stores: `design/data-model.md`.

9 Tech stack and repo layout

CONCERN	CHOICE
Language / build	TypeScript (strict), Vite
UI	React, CSS modules, inline SVG for the car diagram
Charts	uPlot (small, fast for streaming)
Storage	IndexedDB via `idb`
Hardware	Web Serial API (`navigator.serial`)
Tests	Vitest (unit, simulator-driven integration), Playwright (UI against demo car)
PDF export	Print stylesheet + `window.print()` to start
Hosting	Static site (GitHub Pages) — HTTPS satisfies Web Serial

```
car-diagnostic-analyser/
├─ design.html · design.pdf · README.md
├─ design/                  this design's detail docs + wireframes
├─ src/
│  ├─ transport/            ElmSerialTransport, SimulatorTransport
│  ├─ protocol/             obd2/, uds/, isotp, dtc, pid decoders
│  ├─ guard/                ReadOnlyGuard + allowlist
│  ├─ scan/                 discovery, planner, runner, events
│  ├─ profiles/             generic-obd2.json, vehicles/*.json
│  ├─ data/                 IndexedDB repositories
│  └─ ui/                   screens/, car-diagram/, components/
├─ sim/scenarios/           demo-car and test scenarios (*.json)
└─ tests/
```

10 Testing and the demo car

- **Decoders**: table-driven unit tests from published PID formulas and real captured replies.
- **Guard**: every blocked service has a test proving it never reaches the transport.

- **Simulator scenarios**: JSON that maps requests to replies with latency and failure injection. The same files drive tests and the demo car. Scenarios: healthy car, lean engine + low tyre + brake check due, ABS offline, old K-line car, flaky adapter (random `NO DATA`, mid-scan disconnect).
- **Replay**: every real scan's raw log can be turned into a scenario, so bugs found in the car become regression tests.
- **On the car**: first real test only after the simulator suite passes; parked, ignition on, engine off, then at idle.

---

11 Roadmap

Start with the basics on one car and one adapter, then extend one system at a time. Each phase ends with something demoable. Acceptance criteria per phase are in `design/roadmap.md`.

Phase 1	Project skeleton, SimulatorTransport, scan engine with a fake plan, progress UI
Phase 2	ElmSerialTransport over Web Serial; adapter init and identification (`ATI`, `ATRV`); ReadOnlyGuard
Phase 3	Discovery: protocol, VIN, supported-PID bitmaps, battery voltage
Phase 4	Engine fault codes: stored, pending, permanent; DTC descriptions
Phase 5	Check-engine light and readiness monitors
Phase 6	Freeze-frame data for stored codes
Phase 7	Live data gauges and charts
Phase 8	Animated car diagram during the scan — **first release (v0.1)**
Phase 9	Save scans to IndexedDB, history list
Phase 10	Compare two scans
Phase 11	Export report (print/PDF)
Phase 12	Maintenance & inspection log feeding the report
Phase 13	Vehicle profile system; first profile for one make/model/year
Phase 14	ABS module fault codes via the profile
Phase 15	Per-wheel tyre pressures where the car exposes them
Phase 16	Oil-life / service-due data where supported

12 Risks and open questions

RISK	MITIGATION
Clone ELM327 adapters with buggy firmware or wrong baud	Recommend an STN-based adapter; baud auto-probe; tolerant parser; “flaky adapter” scenario in tests
Manufacturer data (ABS, TPMS, oil life) is undocumented	Profiles are data files; start with one car you own; label everything by source
Newer cars block diagnostics behind a security gateway	Generic OBD-II reads usually still work; show “module not reachable” honestly
Users read the report as a safety certificate	Wording and manual-inspection items make clear what was *not* checked
Web Serial is Chromium-desktop only	Detect and explain on unsupported browsers; demo car still works everywhere
Adapter left plugged in drains the battery	Reminder on scan finish

Open questions

- Which car is the first target for an enhanced profile (make, model, year, TPMS type)?
- Which adapter to buy first — OBDLink EX, or reuse an existing ELM327?
- DTC description source: a bundled generic P0xxx list is easy; manufacturer-specific codes need per-profile tables.
- Units: metric by default (°C, bar, km) with a setting for psi / miles?